Exploratory Testing Planner Agent

Operator Guide - Start, Run & Verify

A step-by-step guide to starting the stack, running the full testing suite on the emulator, verifying every feature, and understanding what each capability means.

Scope: ETA-REQ-301 to 308 (Work Packages WP1 - WP9)
Autonomous, self-learning GraphRAG test engineer over Neo4j + a live Android app.
Assumes the emulator, Neo4j, and the .env file are already set up.

Contents

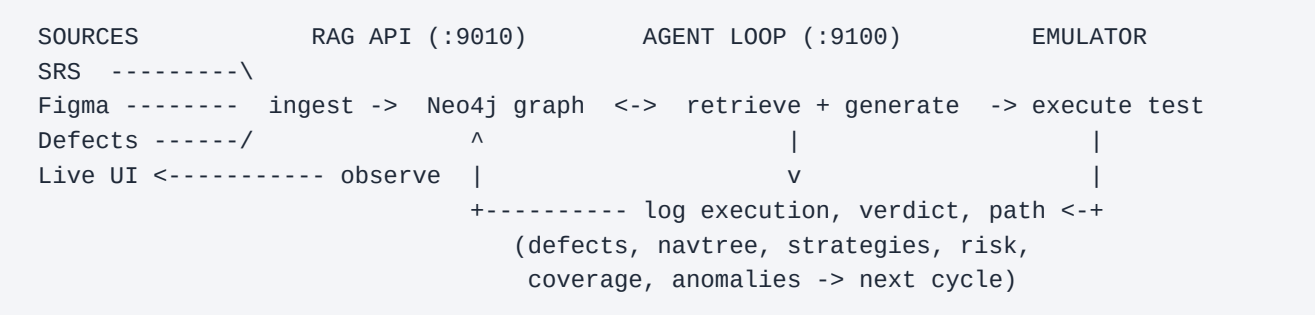
1. System Overview & Architecture

The agent turns whatever knowledge exists about an app - a requirements spec (SRS), a UI design export (Figma), a defect history, and above all the LIVE running app itself - into high-value exploratory tests. Every execution feeds results back into a Neo4j knowledge graph, so the agent gets measurably smarter each cycle. No single source is required: with nothing but an installed app, it explores the device and builds its own map.

The moving parts

`Neo4j`	The knowledge graph (bolt://localhost:7687). Stores SRS, UI model, defects, navigation memory, execution logs, strategies, risk and anomalies. Already set up.
`RAG API :9010`	rag_api.main - the graph service. Ingestion, hybrid retrieval, and all learning endpoints. Talks to Neo4j.
`Gateway :9100`	gateway.main - the thin public router + the agent loop (LangGraph). Serves the operator dashboard and proxies to the RAG API.
`Emulator + mobilerun`	The Android device under test. The executor drives it, observes each screen, and executes generated test steps. Already set up.
`Dashboard`	http://localhost:9100/dashboard - a single live view of everything the agent knows and is doing.

Data flow (one cycle)



Because the loop writes back what it learns (navigation paths, which strategies find bugs, regression risk, emerging anomalies), the next test is generated with more context than the last.

2. Prerequisites (already set up)

This guide assumes the following are already configured on your machine. You only need to confirm they are running.

- **Neo4j:** a local instance is started and reachable at bolt://localhost:7687 with the credentials in .env (NEO4J_URI / NEO4J_USER / NEO4J_PASSWORD).
- **Emulator:** an Android emulator (or device) is running and visible to adb, with the target app installed (TARGET_APP_PACKAGE in .env).
- **Python venv:** the project virtual environment .venv_py_3_13 exists with dependencies installed.
- **.env:** present in the project root with model backend, embeddings, project name, and paths configured.

> **NOTE**
> All commands below are run from the project root: d:\Projects\Exploratory-Testing-Planner-Agent . The Python interpreter is referenced as .venv_py_3_13\Scripts\python.exe (Windows). On Git Bash use forward slashes.

3. Part 1 - Starting the Stack

Start the two services in two separate terminals. Keep them running while you work.

Step 1 - Confirm Neo4j is up

Quick connectivity check (should print NEO4J OK):

```
.venv_py_3_13\Scripts\python.exe -c "from neo4j import GraphDatabase; import os; from dotenv impor
```

Step 2 - Start the RAG API (terminal 1)

```
.venv_py_3_13\Scripts\python.exe -m uvicorn rag_api.main:app --port 9010
```

Wait for startup, then confirm health (should return status ok):

```
curl http://localhost:9010/health
```

Step 3 - Start the Gateway (terminal 2)

```
.venv_py_3_13\Scripts\python.exe -m uvicorn gateway.main:app --port 9100
```

Confirm it is up:

```
curl http://localhost:9100/health
```

Step 4 - Open the dashboard

Open a browser to the live operator dashboard. Pass the project name that matches PROJECT in your .env (default contacts-app):

```
http://localhost:9100/dashboard?project=contacts-app
```

The dashboard auto-refreshes every 4 seconds. It will be mostly empty until you ingest knowledge and/or run a testing loop.

Step 5 (optional) - Ingest knowledge sources

This loads the SRS, Figma UI, and sample defects for the project into the graph. It is OPTIONAL - the agent also works with no documents at all (it explores the live app). Paths come from SRS_PATH / FIGMA_PATH in .env.

```
.venv_py_3_13\Scripts\python.exe scripts/ingest_all.py
```

> **NOTE**
> Ingesting the SRS uses the configured model backend to extract requirements and business rules, so it consumes model tokens (unless you use a local/ngrok backend). Ingesting Figma and defects does not.

4. Part 2 - Running the Full Testing Suite on the Emulator

There are three runner scripts in clients/. Each drives the same agent loop; they differ in whether a real device is involved.

`executor_runner.py`	THE MAIN ONE. Generates a test, EXECUTES it on the emulator via mobilerun, captures the real trajectory (screens + steps), self-heals on failure, and logs everything back. This is the full testing suite.
`crawl_runner.py`	Autonomous explorer. Maps the app into the Live App Model BEFORE testing - useful for a zero-doc app so the agent has a map to reason over.
`simulator_runner.py`	No device. Runs the planner/learning loop but SIMULATES pass/fail verdicts. Use it to exercise generation + learning without an emulator.

Recommended run order

Step 1 (optional) - Crawl the app to seed the Live App Model

Drives the emulator to explore the app and record its screens/transitions. Controlled by CRAWL_ROUNDS and CRAWL_MAX_STEPS in .env.

```
.venv_py_3_13\Scripts\python.exe clients/crawl_runner.py
```

Step 2 (optional) - Ingest documents

If you have an SRS / Figma / defects for the app, run the ingest from Part 1, Step 5. Skip for a pure zero-doc exploration.

Step 3 - Run the executor loop on the emulator

This is the main testing suite. It runs EXECUTOR_ROUNDS rounds; each round the agent generates one test and executes it on the device against TARGET_APP_PACKAGE.

```
.venv_py_3_13\Scripts\python.exe clients/executor_runner.py
```

Watch it live in two places while it runs:

- **Dashboard:** the Live Activity strip shows the current test and a verdict stream; the App Model graph grows as new screens are discovered; risk, coverage and anomalies update each round.
- **Console / logs:** mobilerun's on-device 'thinking' and actions are teed to logs/mobilerun.log and streamed into the dashboard log view.

Key configuration knobs (.env)

`PROJECT`	The project name (graph namespace). All runners and the dashboard must agree on it.
`TARGET_APP_PACKAGE`	The Android package under test, e.g. com.android.contacts.
`EXECUTOR_ROUNDS`	How many generate-and-execute rounds the executor runs (default 2).
`EXECUTOR_MAX_STEPS`	Max on-device steps mobilerun may take per test.
`SELF_HEAL`	1 = classify failures and attempt an adaptive recovery + retry (WP7); 0 = off.
`SIM_ROUNDS / SIM_FAIL_EVERY`	For simulator_runner: number of rounds and how often to force a simulated failure.

`MODEL_BACKEND`	gemini | openrouter | ngrok - which LLM generates the tests.
`EMBEDDING_BACKEND`	fastembed (local, free) recommended - powers semantic retrieval & dedup.

How many tests does a run produce?

One test per round. executor_runner runs EXECUTOR_ROUNDS tests on the device; simulator_runner runs SIM_ROUNDS simulated tests. Increase the rounds to build up more history (the learning signals get richer with more executions).

5. Part 3 - Running the Verification Suite (101 checks)

Before (or instead of) a live emulator run, you can prove that every feature works end to end with a single LLM-free script. It seeds throwaway projects with synthetic data and asserts that each endpoint returns correct graph-derived results. No emulator and no model tokens required - only the RAG API (:9010) and Neo4j.

```
.venv_py_3_13\Scripts\python.exe scripts/verify_enhancements.py
```

Expected final line:

```
RESULT: 101 passed, 0 failed
```

The output is grouped by feature (REQ-301 Defect History, REQ-302 Navigation Tree, ... WP8 anomaly detection, WP9 live observability). A PASS line for each acceptance criterion is your proof that the corresponding capability is wired correctly.

> **NOTE**
> This is the fastest way to confirm a healthy install. If all 101 pass, the backend, graph schema, learning loop, and dashboard data contract are all correct.

6. Part 4 - Manual Feature Checks

Each capability below lists: what it MEANS, how to CHECK it by hand (an endpoint on the RAG API :9010), and what a healthy RESULT looks like. Replace PROJECT with your project name. These use GET unless noted; use a browser, curl, or the Swagger UI at http://localhost:9010/docs .

Live App Model (self-built UI map) [WP1]

MEANS

The agent builds its own map of the app by observing each running screen. Distinct screens become UIState nodes; navigations between them become transitions. Re-seeing a screen does not duplicate it (deduped by a structural signature); reaching a new screen adds a node. Stale screens fade after app updates (knowledge decay).

```
curl "http://localhost:9010/appmodel/graph?project=PROJECT"
```

HEALTHY RESULT

A JSON object with nodes (each screen with label, visit_count, has_shot) and edges (action-labelled transitions). state_count grows as new screens are found; revisits only bump visit_count. On the dashboard this renders as the App Model graph you can watch grow.

Defect Intelligence [WP2 / REQ-301]

MEANS

Historical defects are ingested as first-class knowledge. The agent computes which feature areas are most defect-prone and biases test generation toward them - broken areas tend to break again.

```
curl "http://localhost:9010/defects/summary?project=PROJECT"
curl "http://localhost:9010/defects/prone-areas?project=PROJECT"
```

HEALTHY RESULT

total_defects, unresolved_defects, a severity distribution, and a ranked prone_areas list. Generated tests should skew toward the highest-density areas versus a no-defect baseline.

Execution-trace capture [WP3 / REQ-303.1]

MEANS

Every test run is persisted with its verdict, timing, step counts, error type, and the exact ordered path of screens it walked. This is the raw material for all experiential learning.

```
curl "http://localhost:9010/execution/logs?project=PROJECT&limit=10"
```

HEALTHY RESULT

One ExecutionLog per run with verdict, duration_ms, device_steps, error_type, and path_labels. After a real executor loop there is one per round.

Navigation memory (NavTree) [WP4 / REQ-302]

MEANS

The agent remembers the shortest successful route to each screen and reuses it instead of re-exploring; routes that repeatedly fail are flagged 'avoid'.

```
curl "http://localhost:9010/navtree/retrieve-path?project=PROJECT&screen=SCREEN_LABEL"
curl "http://localhost:9010/navtree/failed-paths?project=PROJECT"
```

HEALTHY RESULT

retrieve-path returns an ordered step list (the proven shortest path). failed-paths lists dead-ends with a success/visit ratio below the avoid threshold. Re-running a test reuses the stored path, so it takes fewer exploratory steps.

Experiential learning + business logic [WP5 / REQ-303 + BLI]

MEANS

Recurring failures are mined into ErrorPatterns with suggested mitigations; test STRATEGIES that find defects gain effectiveness score; a COVERAGE heatmap tracks what is tested; SESSIONS give continuity; and SRS extraction is multi-pass with confidence, provenance, versioning and drift detection (re-ingesting a changed SRS flags the affected areas for re-test). Stale knowledge decays (90-day half-life).

```
curl "http://localhost:9010/execution/error-patterns?project=PROJECT"
curl "http://localhost:9010/strategy/memory?project=PROJECT"
curl "http://localhost:9010/coverage/heatmap?project=PROJECT"
curl "http://localhost:9010/srs/drift?project=PROJECT"
```

HEALTHY RESULT

error-patterns lists recurring failure signatures + mitigations; strategy/memory lists strategies with a decay-weighted effectiveness score; the heatmap reports covered vs total areas/requirements; srs/drift reports the SRS version and any areas needing re-test.

Multi-dimensional knowledge [WP6 / REQ-304]

MEANS

Knowledge can be partitioned by profile / platform / application so retrieval only returns in-dimension context, and tests from one environment can be SUGGESTED for another (cross-dimensional transfer with a confidence score).

```
curl "http://localhost:9010/dimensions/list?project=PROJECT"
curl "http://localhost:9010/dimensions/transfer-suggestions?project=PROJECT&platform=android"
```

HEALTHY RESULT

dimensions/list shows the registered profile/platform/application values. Retrieval with a dimension filter excludes out-of-dimension content (no leakage). transfer-suggestions proposes same-app tests for an untested environment with a transfer_confidence.

Self-healing + regression risk [WP7 / REQ-305, 306]

MEANS

On failure the executor classifies the cause (navigation, element-not-found, assertion, timeout, crash, permission) and attempts a category-appropriate recovery + retry, logging the outcome. Separately, a regression-risk score per area (from defect density + failure ratio + recency + navigation instability) biases generation toward fragile areas.

```
curl "http://localhost:9010/risk/scores?project=PROJECT"
```

HEALTHY RESULT

risk_scores ranks areas by regression_risk_score in [0,1] with the contributing factors exposed (defect_density, fail_ratio, defect_recency, nav_instability). In a live run, a recoverable failure shows a recovery_action ending in RECOVERED on its ExecutionLog.

Quality metrics + semantic dedup + anomalies [WP8 / REQ-307, 308]

MEANS

Per-test effectiveness (defect-discovery rate, execution stability, coverage contribution) is tracked. Duplicate detection uses embedding-cosine similarity, not just word overlap, so reworded duplicates are caught. An anomaly engine scans execution logs for emerging issues (failure-rate spikes, execution-time regressions, new error types, navigation instability) and surfaces them so the agent generates targeted investigation tests.

```
curl "http://localhost:9010/tests/effectiveness?project=PROJECT"
curl -X POST "http://localhost:9010/anomalies/detect" -H "Content-Type: application/json" -d '{"pr
curl "http://localhost:9010/anomalies?project=PROJECT"
```

HEALTHY RESULT

effectiveness returns per-test metrics; a reworded-but-equivalent test title scores a high similarity on /tests/dedup-check while an unrelated one scores low; anomalies/detect returns current alerts (type, area, severity, description) which then bias the next generated test.

Live observability [WP9]

MEANS

A single dashboard shows what the agent is doing now (current test + live verdict stream), the growing App Model graph, and 'getting smarter' trend charts (cumulative bugs, states discovered, steps-per-run, pass-rate).

```
curl "http://localhost:9010/session/live?project=PROJECT"
curl "http://localhost:9010/metrics/trends?project=PROJECT"
```

HEALTHY RESULT

session/live reports status (executing/idle), the most-recent test, and a recent verdict stream; metrics/trends returns per-run series. Both drive the dashboard's top strip and trend charts.

7. Part 5 - Dashboard Walkthrough

Open http://localhost:9100/dashboard?project=PROJECT . One poll drives every panel. From top to bottom:

- **Header pills:** project, model backend, a live status pill (executing / idle), the active session, and its dimension tags.
- **KPI tiles:** total tests, pass rate, bugs found, coverage, top regression risk, defects in the knowledge base, self-heals, anomalies, requirements.
- **Live Activity strip:** the test running right now (or the last one) with its verdict, plus a colored verdict stream of recent rounds.
- **Execution:** the test-case table and an Execution Timeline (verdict, duration, steps, classified error type, and self-healing outcome per run).
- **Risk & Defects:** regression-risk heat meters, defect intelligence (severity + prone areas), and live bugs found.
- **Learning:** strategy memory (effectiveness bars), error patterns (+ mitigations), and navigation memory (nodes, dead-ends, paths to avoid).
- **Quality & Anomalies:** anomaly alerts by severity, and per-test effectiveness metrics.
- **Getting Smarter - Trends:** sparkline charts proving improvement over runs.
- **Knowledge:** SRS stats, business-logic health (versions, drift, low-confidence rules), and dimensions.
- **Live App Model:** the force-directed screen graph (hover a node for its screenshot) plus a screenshot card grid. Watch it grow as the executor explores.

> **NOTE**
> If a panel is empty, that source simply has no data yet for this project - run an ingest or an executor loop. Panels degrade independently; one empty source never blanks the board.

8. Part 6 - Feature Glossary (WP1 - WP9)

WP1 Live App Model	Self-built, screenshot-grounded UI state graph; the always-available knowledge source for zero-doc apps.
WP2 / REQ-301 Defects	Defect history as a knowledge source; biases tests toward defect-prone areas.
WP3 / REQ-303.1 Traces	Persist the executor's real trajectory + rich verdict per run.
WP4 / REQ-302 NavTree	Remember shortest successful paths; reuse them; avoid failed ones.
WP5 / REQ-303 Learning	Error patterns, strategy memory, coverage heatmap, sessions, knowledge decay, and evolving business-logic extraction (multi-pass, confidence, provenance, versioning, drift).
WP6 / REQ-304 Dimensions	Partition + filter knowledge by profile/platform/application; cross-dimensional transfer.
WP7 / REQ-305,306	Self-healing recovery on failure; regression-risk-weighted prioritization.
WP8 / REQ-307,308	Test effectiveness metrics, embedding-based semantic dedup, and anomaly detection from execution patterns.
WP9 Observability	The operator dashboard: live status, verdict stream, App Model graph, and gets-smarter trend charts.

9. Troubleshooting

RAG API won't start / ServiceUnavailable	Neo4j is not running or the URI/credentials in .env are wrong. Start Neo4j and re-run the Step-1 check.
Dashboard banner: cannot reach gateway	The gateway (:9100) is not running, or you opened the dashboard on the wrong port. Start gateway.main.
Panels are empty	No data for this project yet. Confirm the ?project= in the URL matches PROJECT in .env, then ingest or run a loop.
UnicodeEncodeError on Windows console	The runners already force UTF-8; if you wrapped them, set PYTHONUTF8=1.
Executor: no device / adb	Ensure the emulator is running and 'adb devices' lists it before starting executor_runner.py.
Semantic dedup shows enabled=false	EMBEDDING_BACKEND is off/unavailable. Set it to fastembed (local, no key required).

10. Appendix - Endpoint Reference

All on the RAG API (:9010) unless noted. The Gateway (:9100) mirrors the agent-facing ones and serves the dashboard. Full interactive docs: http://localhost:9010/docs .

`GET /health`	Service + Neo4j status.
`POST /ingest/srs | /ingest/figma | /ingest/defects`	Load knowledge sources.
`POST /retrieve`	Hybrid (vector+keyword+graph) context retrieval, dimension-filterable.
`POST /agent/next-testcase (gateway)`	Generate the next exploratory test.
`POST /agent/log-verdict-and-next (gateway)`	Log a verdict and get the next test.
`POST /liveui/observe | GET /appmodel/graph`	Record an observed screen | read the UI map.
`GET /defects/summary | /defects/prone-areas`	Defect intelligence.
`GET /navtree/retrieve-path | /navtree/failed-paths`	Navigation memory.
`GET /execution/logs | /execution/error-patterns`	Execution history + mined patterns.
`GET /strategy/memory | /coverage/heatmap`	Strategy scores | coverage snapshot.
`POST /session/start | /session/end ; GET /session/context | /session/live`	Sessions + live status.
`GET /srs/drift | /business-logic/rules`	SRS versioning/drift | extracted rules.
`GET /dimensions/list | /dimensions/transfer-suggestions`	Dimensions + transfer.
`GET /risk/scores`	Regression risk per area.
`GET /tests/effectiveness ; POST /tests/dedup-check`	Effectiveness metrics | semantic dedup.
`POST /anomalies/detect ; GET /anomalies`	Detect + read anomaly alerts.
`GET /metrics/trends`	Gets-smarter trend series.
`GET /dashboard | /dashboard/data (gateway)`	The dashboard page | its aggregated payload.